

## Proyecto 2

### **Mapeando el Caos: Indexación y Organización de Datos No Estructurados para Datos Multimedia.**

#### **1- Enunciado**

El objetivo del proyecto es comprender y aplicar algoritmos de búsqueda y recuperación de información basados en el contenido de los objetos. El proyecto debe integrar de manera eficiente técnicas de indexación y búsqueda por similitud, articulándose con el desarrollo realizado en la primera entrega, con el fin de construir una base de datos multimodal funcional.

El desarrollo se divide en dos componentes principales:

- a) Diseño y construcción eficiente de un índice invertido para búsqueda y recuperación de información en documentos textuales.
- b) Diseño y construcción eficiente de un índice para recuperación de contenido multimedia (imágenes o audio), utilizando descriptores locales.

Para validar el funcionamiento del sistema, se deberán emplear datasets reales, preferentemente obtenidos de plataformas como Kaggle o GitHub. El proyecto deberá demostrar eficiencia en las operaciones, una estructura de código clara y organizada, así como incluir una breve documentación técnica que describa el diseño personalizado de las técnicas, las decisiones adoptadas y los resultados experimentales obtenidos.

#### **2- Construcción del Índice Invertido Textual (Full-Text Search)**

Se implementará un índice invertido que de soporte eficiente a la recuperación por ranking orientado a consultas en lenguaje natural. Considere los siguientes pasos generales:

##### **a) Preprocesamiento del Texto:**

Para cada fila de la tabla, se concatenarán los campos textuales en un solo bloque de texto. Luego, se aplicarán las siguientes técnicas de limpieza y normalización:

- Tokenization
- Filtrar Stopwords
- *Eliminar signos innecesarios*
- Reducción de palabras (*Stemming*)

##### **b) Construcción del Índice en Memoria Secundaria**

Se construirá un índice invertido optimizado para grandes volúmenes de datos, siguiendo estos pasos:

- **Cálculo de TF-IDF:** Para cada término en cada documento, se calculará su peso usando la fórmula TF-IDF, y se almacenará en el índice.
- **Norma del documento:** Se calculará una sola vez la norma euclidiana de cada documento y se almacenará para acelerar el cálculo de similitud.
- **Indexación en memoria secundaria:** Para colecciones extensas, se implementará el algoritmo **SPIMI** (Single-Pass In-Memory Indexing), que permite construir el índice en bloques sin necesidad de cargar toda la colección en memoria RAM.
  - En el proceso de MergeBlocks **se debe usar toda la memoria RAM disponible (B buffers)**

- El informe debe incluir una explicación gráfica del funcionamiento de SPIMI, detallando su implementación paso a paso.

**c) Consulta**

- La consulta es una frase en lenguaje natural.
- El scoring se obtiene aplicando la similitud de coseno sobre el índice invertido en **memoria secundaria**.
  - Evitar cargar todo el índice en la RAM.
- La función de recuperación debe retornar el top-k de documentos más similares a la consulta, ordenados por el score.

**3- Indexación de descriptores locales (Multimedia Database)**

Se implementará un sistema de indexación basado en descriptores locales que permita la recuperación eficiente de objetos multimedia. Previamente, se recomienda que la información estructurada de los objetos multimedia (metadata) se almacene en forma de tabla siguiendo las estrategias del Proyecto 1. Considere los siguientes pasos generales:

**a) Extracción de características**

Se utilizarán librerías especializadas para extraer un grupo de vectores característicos (descriptores locales) de cada objeto multimedia.

- Tutorial para extraer características en imágenes:
  - [SIFT Algorithm](#)
  - [Inception v3](#)
  - [ResNet50](#)
- Tutorial para extraer características en audio:
  - Se puede usar MFCC (Mel-Frequency Cepstral Coefficients) o espectrogramas
  - [MFCC Feature Extraction from Audio](#)

**b) Construcción del diccionario visual o acústico (Codebook)**

- Se recopilan los descriptores locales extraídos de múltiples objetos (imágenes o audios) para conformar un único conjunto de datos.
- Se aplica el algoritmo K-Means para agrupar los descriptores en k clusters, donde cada cluster representa un “visual Word” (imágenes) o un “acoustic word” (audios).
- El centroide de cada cluster se denomina codeword.
- El conjunto de codewords forma el diccionario visual o acústico de la colección. Será la base para representar objetos como histogramas.

**c) KNN Secuencial**

- Representar cada objeto de la colección mediante un histograma de visual words o acoustic words, utilizando el diccionario previamente construido.
- Para el objeto de consulta, genera su correspondiente histograma de visual/acoustic words.
- Aplicar técnicas de ponderación como TF-IDF para asignar un peso a cada palabra (visual o acústica), según su importancia relativa en el conjunto de datos.
- Calcular la similitud de coseno entre el histograma del objeto de consulta y los histogramas almacenados.

- Recuperar los K objetos más similares en función de la similitud obtenida. Debe estar optimizando con una cola de prioridad (heap) para mantener de forma eficiente los K resultados más relevantes durante la búsqueda.

**d) KNN con Indexación Invertida**

- Cada descriptor local se asigna al codeword más cercano del diccionario.
- Construir una estructura de índice invertido, donde cada palabra (visual o acústica) referencia los objetos en los que aparece.
- Aplicar TF-IDF para estimar la relevancia de cada palabra en cada objeto.
- El algoritmo de búsqueda debe seguir un enfoque análogo al utilizado en los índices invertidos textuales, adaptado al dominio visual o acústico, permitiendo una recuperación eficiente del Top-K objetos más relevantes.

#### **4- FrontEnd**

Se desarrollará una interfaz gráfica que permita interactuar con los sistemas de recuperación de información textual y multimedia, facilitando la ejecución de consultas y la visualización de resultados de forma clara y eficiente.

**a) Para búsqueda en texto:**

Se debe construir una interfaz que permita consultar el índice invertido textual mediante una sintaxis estructurada. Las funcionalidades mínimas incluyen:

- Entrada de consultas textuales mediante una sintaxis tipo SQL, o bien una sintaxis personalizada definida por el grupo.
  - **Ejemplo** de consulta con sintaxis SQL:

```
SELECT title, artist, lyric
FROM Audio
WHERE lyric @@ 'amor en tiempos de guerra'
LIMIT 10;
```

- Opción para especificar la cantidad de documentos a recuperar (**Top-K**).
- Presentación de resultados de manera clara y amigable para el usuario:
  - Mostrar los campos solicitados (por ejemplo, título, artista, fragmento de letra).
  - Indicar el **tiempo de respuesta** de la consulta.

**b) Para búsqueda en datos multimedia**

Se debe construir una interfaz que permita realizar consultas por similitud utilizando objetos multimedia como entrada (imagen o audio). Las funcionalidades mínimas son:

- Opción para cargar el archivo multimedia de consulta y ejecutar la búsqueda basada en su contenido.
- Posibilidad de utilizar una sintaxis tipo SQL o definida por el grupo para consultas multimodales.
  - **Ejemplo** de consulta con sintaxis SQL:

```
SELECT id, title
FROM Multimedia
WHERE image_sim <-> 'D:\imagenes\query.jpg'
LIMIT 10;
```

- Presentación interactiva de los resultados de la búsqueda, incluyendo:
  - Visualización de los objetos similares (imágenes o audios con sus respectivos metadatos).
  - Asociación con resultados textuales relacionados (por ejemplo, título o descripción).
  - Indicación del **tiempo de ejecución** de la consulta.

## 5- Experimento y Evaluación de Desempeño:

### a) Búsqueda en texto:

Evalúe el rendimiento de su implementación del índice invertido comparándolo con los resultados obtenidos utilizando PostgreSQL. Para ello:

- Diseñe consultas equivalentes en ambos sistemas y mida el **tiempo de respuesta** y la **calidad de los resultados** ¿Se obtiene los mismos resultados? (Fundamente en el informe).
- Investigue y explique cómo PostgreSQL realiza la recuperación de información textual:
  - El tipo de índice que utiliza (por ejemplo, GIN o GiST).
  - La función de similitud o ranking que emplea (ts\_rank, ts\_rank\_cd, entre otras).
  - Cómo gestiona el procesamiento de consultas en campos tsvector y tsquery.

|           | MyIndex | PostgreSQL |
|-----------|---------|------------|
| N = 1000  |         |            |
| N = 2000  |         |            |
| N = 4000  |         |            |
| N = 8000  |         |            |
| N = 16000 |         |            |
| N = 32000 |         |            |
| N = 64000 |         |            |
| ...       |         |            |

### b) Búsqueda en bases de datos multimedia (imágenes o audio)

Ejecute y compare los algoritmos **KNN secuencial** y **KNN con indexación invertida** sobre una colección de objetos multimedia de tamaño N, evaluando su eficiencia en función del tiempo de ejecución.

Además, compare los resultados con una implementación en **PostgreSQL** utilizando extensiones diseñadas para búsquedas vectoriales. Considere los siguientes aspectos:

- La **eficiencia** en consultas de similitud sobre vectores de características (por ejemplo, descriptores de imagen o audio).
- El impacto de la **dimensionalidad** en el rendimiento de los índices (considerar la **maldición de la dimensionalidad**).

### Herramientas recomendadas para la comparación:

1. [pgVector](#): Extensión de PostgreSQL para búsqueda por similitud sobre vectores (open-source).

2. [Faiss](#): Librería desarrollada por Facebook para búsqueda eficiente de vectores, con soporte para CPU/GPU y algoritmos como **HNSW**.

|           | KNN-Secuencial | KNN-Indexado | KNN- PostgreSQL |
|-----------|----------------|--------------|-----------------|
| N = 1000  |                |              |                 |
| N = 2000  |                |              |                 |
| N = 4000  |                |              |                 |
| N = 8000  |                |              |                 |
| N = 16000 |                |              |                 |
| N = 32000 |                |              |                 |
| N = 64000 |                |              |                 |

\* Mantener el valor de K = 8.

#### 4. Datasets

- [Fashion Product Images Dataset \(kaggle.com\)](#)
- [Audio features and lyrics of Spotify songs Dataset \(kaggle.com\)](#)
- [FMA: A Dataset For Music Analysis \(github.com\)](#)

#### 5. Informe del proyecto y Rúbrica de evaluación

El proyecto debe documentarse adecuadamente mediante un informe técnico conciso y bien redactado, acompañado por un repositorio en GitHub que contenga el código, ejemplos de uso y documentación básica.

- El informe puede redactarse en el mismo **README** o **Wiki** de GitHub.
- Cuidado en la **ortografía, redacción técnica y consistencia de los párrafos**.
- El informe no debe ser extenso, pero sí debe cubrir **todos los aspectos relevantes de la implementación**.

#### Estructura del Informe y Rúbrica

| Sección                                                 | Contenido esperado                                                                                                                                                                                                                                                                                      | Puntaje |
|---------------------------------------------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|---------|
| <b>[1 punto] Introducción</b>                           | <ul style="list-style-type: none"> <li>- Descripción del dominio de datos (texto, imágenes o audio).</li> <li>- Justificación de la necesidad de una <b>base de datos multimodal</b> para tareas de recuperación por contenido.</li> </ul>                                                              | 1       |
| <b>[7 puntos] Backend – Índice Invertido para Texto</b> | <ul style="list-style-type: none"> <li>- <b>Construcción del índice invertido en memoria secundaria.</b></li> <li>- Ejecución eficiente de consultas utilizando <b>Similitud de Coseno</b>.</li> <li>- Explicación del mecanismo de construcción de índices invertidos en <b>PostgreSQL</b>.</li> </ul> | 7       |

|                                                                        |                                                                                                                                                                                                                                                                                                                                                                                                                                              |   |
|------------------------------------------------------------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|---|
| <b>[6 puntos] Backend – Índice Invertido para Descriptores Locales</b> | <ul style="list-style-type: none"> <li>- Descripción del proceso de construcción del <b>Bag of Visual Words / Acoustic Words</b>.</li> <li>- Diseño de la técnica de indexación utilizada para organizar los descriptores.</li> <li>- Implementación de la búsqueda KNN sobre los histogramas (secuencial e indexada).</li> <li>- Análisis del impacto de la <b>maldición de la dimensionalidad</b> y estrategias para mitigarla.</li> </ul> | 6 |
| <b>[4 puntos] Frontend</b>                                             | <ul style="list-style-type: none"> <li>- Diseño y usabilidad de la interfaz gráfica (GUI).</li> <li>- Inclusión de un mini-manual de usuario.</li> <li>- Capturas de pantalla que evidencien la funcionalidad del sistema.</li> <li>- Comparación visual con otras soluciones similares (opcional).</li> </ul>                                                                                                                               | 4 |
| <b>[2 puntos] Experimentación</b>                                      | <ul style="list-style-type: none"> <li>- Presentación de resultados experimentales mediante <b>tablas y gráficos comparativos</b>.</li> <li>- Análisis crítico de los resultados obtenidos (rendimiento, precisión, escalabilidad).</li> </ul>                                                                                                                                                                                               | 2 |

## 6. Entregable

- El código fuente del proyecto será alojado en GitHub.
- Para facilitar el despliegue del software deben usar Docker Compose.
- En el Canvas subir solo el **enlace público** del proyecto.
- La fecha límite de la entrega figura en el Anexo del Silabo.
- La **presentación final del proyecto se realizará en clase**, siendo evaluada en función de:
  - Funcionamiento del sistema
  - Claridad expositiva
  - **Capacidad de respuesta ante preguntas del profesor y los compañeros.**